

GLO-7030 : Projet de session

Leopold A. G. Chappuis¹, Alexandre Eberhardt¹, Sophie Goulet¹, Rémi Hercouët¹

¹Département d'informatique et de génie logiciel, Université Laval

leopold-andre-gaston.chappuis.1@ulaval.ca, alexandre.eberhardt.1@ulaval.ca

sophie.goulet.7@ulaval.ca, remi.hercouet.1@ulaval.ca

Abstract—Ce projet étudie la possibilité de prédire des décisions d'ordonnancement Linux à partir de traces `sched_switch` collectées avec `perf`. L'approche principale retenue est un modèle LSTM qui prédit le prochain processus élu sur un CPU à partir d'une fenêtre glissante de décisions précédentes. Le pipeline couvre la collecte des traces, leur tokenisation, l'entraînement, la recherche d'hyperparamètres et l'évaluation.

Le modèle de référence obtient une précision top-1 de 73,22%, une précision top-3 de 96,05% et une précision top-5 de 97,36% sur 294 472 fenêtres de test. Un balayage d'hyperparamètres améliore la précision top-1 jusqu'à 76,60%. Le gain de performance est donc très intéressant.

Un modèle distillé a également été conçu en utilisant ce meilleur modèle comme teacher. La performance obtenue avec ce modèle distillé est très similaire aux modèles LSTM non distillés tout en n'utilisant qu'environ 0,13 million de paramètres, contre 0,97 million pour le baseline initial et 3,50 millions pour la meilleure configuration issue du sweep.

En complément, le projet explore un second cadre expérimental fondé sur un émulateur Python du scheduler Linux et une formulation candidate-based du problème d'ordonnancement, afin de rapprocher l'apprentissage d'une véritable décision de scheduling en boucle fermée.

I. INTRODUCTION

Le scheduler du noyau Linux prend des milliers de décisions d'ordonnancement par seconde, chacune influençant directement la latence, l'équité entre processus et l'utilisation du CPU. Ces décisions reposent sur des heuristiques conçues à la main et affinées sur des décennies, comme CFS ou EEVDF, qui offrent de bonnes garanties générales mais n'optimisent pas explicitement des objectifs comme le temps de réponse ou le turnaround. La difficulté est intrinsèque : le scheduler ne connaît pas à l'avance la durée d'exécution des tâches, plusieurs décisions peuvent être localement raisonnables au même instant, et les objectifs à optimiser sont souvent contradictoires (équité vs. latence, throughput vs. response time). Une question naturelle se pose alors : un modèle appris peut-il reconstruire ces décisions à partir de traces d'exécution, voire les améliorer sur certains critères ?

Depuis Linux 6.6 (Oct. 2023), le scheduler par défaut du noyau pour les tâches normales est l'Earliest Eligible Virtual Deadline First (EEVDF), qui a remplacé le Completely Fair Scheduler (CFS).

Dans ce projet, il faut distinguer deux cadres expérimentaux. Les traces réelles collectées avec `perf` proviennent d'un noyau Linux récent et reflètent donc majoritairement des décisions prises sous EEVDF pour les tâches normales. En revanche, l'émulateur Python et les expériences basées

sur des candidats utilisent CFS comme politique de référence et comme base de comparaison en boucle fermée. Notre hypothèse de travail est qu'un modèle appris peut exploiter l'historique récent d'exécution pour reconstruire, et éventuellement améliorer, certaines décisions observées ou simulées selon le cadre considéré.

Dans ce contexte, notre objectif n'était pas de remplacer immédiatement le scheduler du noyau, mais d'évaluer jusqu'où un modèle appris peut reconstruire, à partir de traces réelles, les décisions d'ordonnancement observées.

Le projet montre une progression en plusieurs étapes. Une première série de travaux met en place un prototype eBPF minimal et une base documentaire. Une deuxième étape introduit un pipeline LSTM supervisé sur des traces `sched_switch`. Enfin, les travaux les plus récents explorent un émulateur Python du scheduler, un apprentissage sur ensembles de tâches candidates et une approche de distillation.

Le présent rapport se concentre donc sur trois contributions concrètes : (i) un pipeline d'apprentissage de type LSTM sur traces réelles, (ii) une optimisation de ce baseline par sweep d'hyperparamètres, et (iii) des extensions expérimentales pour aller au-delà de la simple imitation hors ligne.

II. ÉTAT DE L'ART

Les travaux liés à ce projet se regroupent autour de trois axes : la prédiction de comportement de processus, la prédiction séquentielle des décisions d'ordonnancement, et l'utilisation du renforcement ou de l'imitation pour aller au-delà de la simple reconstruction.

Prédiction de burst et caractérisation de processus. Dès 2005, Negi et Kumar [4] ont proposé d'utiliser des arbres de décision (C4.5) pour apprendre le comportement d'utilisation CPU de processus connus, avec pour objectif de réduire le turnaround time de 1,4% à 5,8% en personnalisant les time slices. Cette approche repose sur des attributs statiques et dynamiques des processus, mais n'intègre pas l'état global du système ni l'historique des décisions récentes. Dans la même veine, Varsha et al. [2] utilisent un moteur ML (Gradient Boosting) pour classifier les processus en catégories de priorité d'ordonnancement à partir de leurs caractéristiques historiques. Ces approches partagent l'idée d'exploiter les attributs de processus, mais restent hors ligne et ne produisent pas de décision d'ordonnancement à proprement parler.

Prédiction séquentielle des décisions. KernelOracle (2025) [3] constitue le travail le plus proche du nôtre dans

ce cadre : un LSTM est entraîné à prédire le prochain processus sélectionné par CFS à partir de traces noyau, sur une machine monoprocesseur. Notre pipeline LSTM s'inscrit dans la même direction, mais l'étend à un contexte multiprocesseur (4 CPUs) et sur un volume de données nettement plus important (1,9 million de fenêtres). La limite commune à ces deux approches est qu'elles imitent les décisions observées sans les évaluer ni chercher à les améliorer.

Au-delà de l'imitation : load balancing, RL et optimisation globale. Chen et al. [1] ont exploré l'utilisation du ML pour le load balancing dans le noyau Linux, ce qui constitue une autre dimension de l'ordonnancement. Shankar [6] offre une vue d'ensemble des applications du ML à l'optimisation du noyau Linux (scheduling, mémoire, énergie, I/O), et souligne que la plupart de ces travaux restent expérimentaux faute d'évaluation en boucle fermée.

Positionnement. Notre contribution se distingue sur deux points. D'abord, contrairement aux approches de prédiction séquentielle (KernelOracle), notre second cadre expérimental formule l'ordonnancement comme un vrai problème de sélection parmi des candidats, avec un état de scheduling riche et une évaluation en boucle fermée face à CFS. Ensuite, plutôt que d'imiter directement CFS ou de recourir au renforcement (coûteux et instable à entraîner), nous utilisons un teacher heuristique multi-critères combiné à une loss listwise, ce qui fournit un signal d'apprentissage plus riche que la cross-entropie sur un seul label.

III. DESCRIPTION DE L'APPROCHE PROPOSÉE

L'approche principale retenue consiste à transformer le problème d'ordonnancement en une tâche de prédiction du prochain processus élu. Les traces `sched_switch` sont converties en séquences de tokens, un token correspondant au nom du processus ordonnancé. Pour chaque CPU, on construit des fenêtres glissantes de taille fixe, et le modèle apprend à prédire le token suivant.

Le modèle de référence est volontairement simple : une couche d'embedding, un LSTM, puis une couche linéaire de classification. Cette simplicité permet de séparer clairement trois questions : la qualité des traces, la capacité du modèle à capturer la dynamique séquentielle, et l'effet des hyperparamètres.

Le pipeline implémenté comprend les étapes suivantes :

- collecte de traces `sched_switch` avec `perf` ;
- extraction du prochain processus élu et tokenisation par CPU ;
- découpage temporel en ensembles d'entraînement, validation et test ;
- entraînement d'un LSTM supervisé ;
- évaluation globale (top-1, top-3, top-5, loss) et analyse par classe.

En plus de cette baseline, d'autres choses ont été expérimentées. Un émulateur Python fidèle aux grandes règles d'un scheduler Linux simplifié, incluant des files CFS et RT, la gestion des réveils, de la préemption et des métriques système, a ensuite servi à construire un dataset plus riche. Dans ce dataset, l'entrée est un état de décision contenant les

tâches candidates, l'historique récent et des caractéristiques globales au lieu d'uniquement une séquence de noms de processus.

Dans cette seconde formulation, l'apprentissage ne repose pas uniquement sur l'imitation directe de CFS par une étiquette unique. Un teacher heuristique attribue plutôt un score continu à chaque tâche candidate à partir de plusieurs critères inspirés du scheduling classique, notamment le temps d'attente, le burst restant, l'équité de type CFS et certaines situations d'urgence. Le modèle est alors entraîné avec une loss combinée qui ancre la prédiction sur le meilleur candidat, tout en apprenant aussi le classement global des candidats et leur comparaison avec la décision CFS de référence. Cette formulation fournit un signal d'apprentissage beaucoup plus riche et ajustable en fonction des objectifs (minimisation des context switches, temps d'exécution) qu'une simple classification du prochain processus élu et rapproche davantage le modèle d'une vraie décision d'ordonnancement.

Dans la configuration candidate-based présentée ici, ce modèle reste relativement compact, avec 90 628 paramètres entraînaibles, ce qui permet un entraînement rapide tout en gardant une architecture plus riche que le baseline séquentiel.

Comme l'illustre la Figure 1, cette seconde architecture combine plusieurs sources d'information : candidat courant, historique récent, ensemble global des tâches et contexte système, avant de produire un score pour chaque tâche exécutable.

Par la suite une distillation est proposée, et entraîne un student compact à partir d'un teacher LSTM plus grand. Globalement, le student a une performance très similaire au teacher tout en utilisant moins de paramètres. Cela permet donc de diminuer le temps requis pour l'inférence, ce qui est essentiel si on souhaite que ce modèle soit en mesure de faire les meilleures prédictions pour le scheduler dans un temps raisonnable.

Autrement dit, l'approche proposée dans le projet évolue selon deux niveaux : (i) une solution opérationnelle et évaluée fondée sur les traces réelles, et (ii) des extensions expérimentales orientées vers des décisions de scheduling plus riches et plus proches d'un déploiement en boucle fermée.

IV. MÉTHODOLOGIE ET EXPÉRIMENTATION

Les expérimentations reposent sur la collecte de traces d'ordonnancement via `perf`, selon le script présenté en Annexe 2. Ces traces sont ensuite converties en un format texte exploitable pour l'analyse (Annexe 3). Ces traces ont été générées sur un système Linux Debian 6.12.48-1, ce qui confirme que seul EEVDF a été utilisé pour obtenir ces traces (détails du système obtenu avec la commande `'uname -a'` en Annexe 4).

Le parsing parcourt le fichier brut ligne par ligne, extrait le champ `next_comm` de chaque événement `sched_switch`, regroupe les événements par CPU, puis construit un vocabulaire en filtrant les tokens de fréquence inférieure à 2. Les séquences tokenisées sont ensuite sauvegardées avec leurs métadonnées.

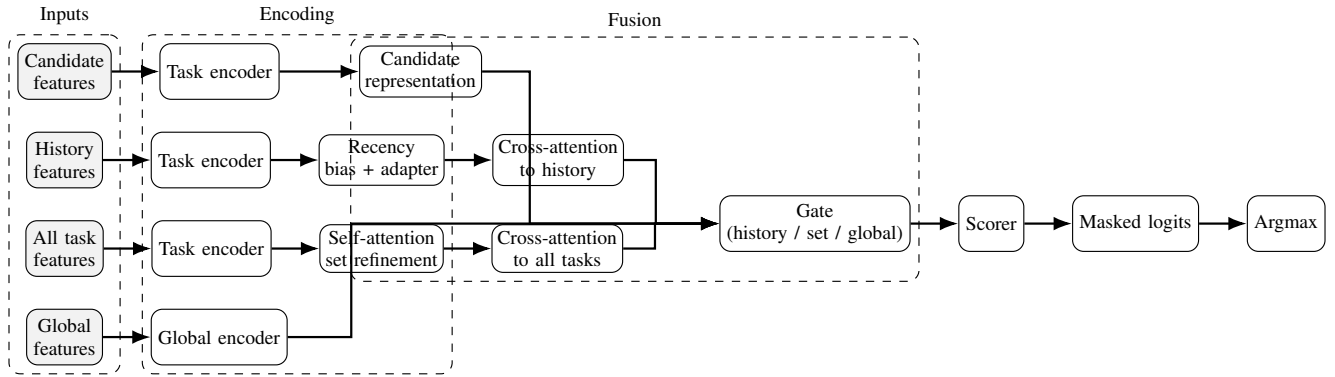


FIG. 1. Vue d'ensemble du modèle candidate-based. Chaque décision est formulée comme un score attribué aux tâches candidates à partir de quatre sources d'information : les candidats eux-mêmes, l'historique récent, l'ensemble global des tâches et le contexte système. Les représentations sont fusionnées par attention et gating, puis converties en logits masqués pour sélectionner une tâche réellement exécutable.

Le dataset principal contient 4 séquences CPU de longueurs respectives 561 814, 539 559, 280 866 et 581 753 événements, pour un vocabulaire final de 129 tokens. Les 50 processus les plus fréquents sont dominés par `stress-ng-iomix`, `swapper/*`, `jbd2/sdal-8`, `stress-ng-cpu`, `hackbench` et plusieurs `kworker`, ce qui reflète bien les charges injectées par le script de collecte.

Les données sont découpées en ensembles d'entraînement, de validation et de test en conservant l'ordre chronologique sur chaque CPU, selon une répartition 70/15/15. Le modèle apprend donc sur le début des séquences et est évalué sur leur continuation, plutôt que sur des traces issues d'autres machines. Avec une fenêtre de 32 événements, cela représente 1 374 665 exemples d'entraînement, 294 471 exemples de validation et 294 472 exemples de test.

Le modèle baseline retenu utilise les paramètres suivants : fenêtre de 32 tokens, embeddings de dimension 128, dimension de l'état caché de 256, 2 couches LSTM, dropout de 0,1, poids de décroissance de 10^{-4} , batch size 1024 et taux d'apprentissage 5×10^{-4} . Cette configuration représente 971 265 paramètres entraînaibles et a été exécutée sur GPU (cuda). L'entraînement est arrêté de manière anticipée si la perte de validation ne s'améliore plus pendant 3 époques consécutives (patience=3).

Une seconde expérimentation a été conduite sous forme de sweep. Elle explore automatiquement plusieurs combinaisons de paramètres à l'aide de `Weights & Biases`, notamment `batch_size`, `lr`, `embed_dim`, `hidden_size`, `num_layers`, `dropout`, `weight_decay` et `window_size`.

La meilleure configuration retenue utilise une fenêtre de 64, une taille cachée de 512, 2 couches, un batch size de 512 et un taux d'apprentissage d'environ 0,00457, pour 3 498 753 paramètres.

En complément du pipeline principal basé sur des traces réelles, le projet repose aussi sur deux méthodologies expérimentales supplémentaires.

La première consiste à utiliser un émulateur d'ordonnancement pour reproduire un environnement contrôlé dans lequel différentes politiques peuvent être comparées dans les mêmes

conditions. Cet émulateur prend CFS comme politique de référence et permet de mesurer, pour chaque exécution, des indicateurs comme le temps d'attente, le temps de réponse, le turnaround, les commutations de contexte et l'équité.

La seconde consiste à reformuler le problème d'apprentissage lui-même : au lieu de prédire uniquement le prochain processus exécuté, on construit à partir de l'émulateur un dataset de décisions candidates, où chaque exemple décrit un état d'ordonnancement, les tâches exécutables à cet instant, l'historique récent et des caractéristiques globales du système. Le modèle est alors entraîné à choisir une tâche parmi cet ensemble de candidats, en combinant un encodage des tâches, une attention sur l'historique récent, un raffinement sur l'ensemble global des tâches et un mécanisme de gating entre ces différentes sources d'information, puis évalué en boucle fermée face à CFS et à une politique heuristique.

Pour cette seconde approche, le dataset d'entraînement a été construit à partir de 50 workloads synthétiques exécutés avec 8 graines aléatoires, soit 400 épisodes de simulation et environ 28 000 décisions d'ordonnancement. Les données sont découpées selon une répartition 70/15/15 au niveau des épisodes, et non des décisions individuelles, afin de limiter les fuites d'information entre entraînement et test. Chaque exemple correspond ainsi à un instant de décision complet produit par l'émulateur sous politique CFS, avec ses tâches candidates, son historique récent et son contexte global.

Enfin, une variante par distillation introduit un cadre d'apprentissage où un modèle student plus compact est entraîné à reproduire les sorties d'un teacher LSTM, afin d'étudier s'il est possible de conserver une bonne qualité de précision avec une architecture plus légère. Le teacher est chargé depuis un checkpoint baseline et ses paramètres sont gelés. Le student est entraîné avec une perte mixte composée d'une cross-entropy sur la cible réelle et d'une divergence KL entre les distributions softmax du teacher et du student, avec une température de distillation $T=4$ et un poids $\alpha=0.5$.

Pour éviter toute ambiguïté, les ordres de grandeur en nombre de paramètres à retenir dans ce rapport sont les suivants : 971 265 pour le LSTM baseline, 3 498 753 pour la meilleure configuration issue du sweep, 125 383 pour le

TABLE I

SYNTHÈSE DES RÉSULTATS PRINCIPAUX. LES DEUX DERNIÈRES LIGNES RELÈVENT D'UN CADRE EXPÉRIMENTAL DISTINCT FONDÉ SUR UN ÉMULATEUR ET NE SONT DONC PAS DIRECTEMENT COMPARABLES AUX TROIS PREMIÈRES.

Expérience	Cadre	Top-1	Top-3	Top-5	Note
LSTM réf.	Traces réelles	73,22%	96,05%	97,36%	0,97 M param.
LSTM + sweep	Traces réelles	76,60%	—	—	3,50 M param.
LSTM distillé	Traces réelles	76,17%	97,50%	98,65%	0,13 M param.
Candidate-based	Émulateur	91,6%	99,9%	—	0,09 M param.
Candidate-based	Émulateur	—	—	—	−10,5% t. réponse vs CFS

student distillé et 90 628 pour le modèle candidate-based.

V. DISCUSSION DES RÉSULTATS

Le Tableau I récapitule les principaux résultats du projet dans un format plus synthétique. Il faut toutefois distinguer les expériences menées sur traces réelles et celles fondées sur l'émulateur : elles n'observent pas le même problème, n'utilisent pas les mêmes entrées et ne doivent donc pas être interprétées comme des comparaisons directes.

Le modèle LSTM de référence obtient des résultats déjà solides sur la tâche de prédiction séquentielle, avec une précision top-1 de 73,22%, une précision top-3 de 96,05% et une précision top-5 de 97,36% sur 294 472 fenêtres de test. Ce premier résultat nous montre que l'historique récent des décisions d'ordonnancement contient bien un signal prédictif exploitable. Autrement dit, le prochain processus élu n'est pas choisi de manière arbitraire : une partie importante de la dynamique observée dans les traces peut être reconstruite à partir de la seule séquence des processus précédemment exécutés.

L'écart marqué entre la précision top-1 et les précisions top-3 et top-5 est également instructif. Il indique que le bon processus est très souvent présent parmi un petit nombre de candidats plausibles, même lorsque le premier choix du modèle n'est pas exactement celui observé dans la trace. Ça suggère que la tâche reste en partie ambiguë au niveau strictement top-1, ce qui est cohérent avec la nature du scheduling Linux : plusieurs décisions peuvent être localement raisonnables à un instant donné, en particulier lorsque plusieurs tâches fréquentes se partagent l'exécution sur une même charge de travail. Le modèle apprend donc mieux un espace restreint de décisions crédibles qu'une reproduction parfaite, événement par événement, de la trace.

L'optimisation par balayage d'hyperparamètres confirme cette lecture. La meilleure configuration testée améliore la précision top-1 jusqu'à 76,60%. En contrepartie, le nombre de paramètres passe d'environ 0,97 million pour le baseline à 3,50 millions pour le meilleur modèle du sweep. Le gain de performance est donc intéressant, mais il s'accompagne bien d'une augmentation nette de la taille du modèle.

La composition même du dataset va dans ce sens. Les traces sont dominées par un petit groupe de processus très fréquents, notamment `stress-ng-iomix`, `swapper/*`, `jbd2/sda1-8`, `stress-ng-cpu`, `hackbench` et plusieurs `kworker`. Il est donc naturel que les meilleures performances globales soient en grande partie tirées par

ces classes dominantes. Les résultats obtenus doivent ainsi être interprétés avant tout comme une bonne capacité de reconstruction de la dynamique observée sur cette famille de charges, et non comme une preuve de généralisation complète à des contextes de scheduling plus variés.

Les résultats obtenus avec le modèle distillé sont très similaires à ceux obtenus avec le baseline. Les précisions top-1, top-3 et top-5 sont respectivement de 76,17%, 97,50% et 98,65%, ce qui est très près des résultats du modèle de référence. De plus, le nombre de paramètres est nettement plus faible, avec environ 0,13 million de paramètres pour le student, contre 0,97 million pour le baseline initial et 3,50 millions pour la meilleure configuration du sweep. Cela permet donc d'obtenir des résultats très similaires avec un modèle beaucoup plus petit. Le temps d'inférence est donc considérablement réduit, ce qui est un avantage important dans la mesure où nous souhaiterions réellement utiliser ce modèle en pratique. La distillation permet ici de transférer la connaissance d'un LSTM plus grand vers un modèle plus léger. Le student présente des performances comparables au baseline, ce qui montre que la distribution adoucie du teacher contient un signal utile. En revanche, ce résultat reste conditionné au fait que le teacher est gelé et que la métrique reste hors ligne. Une étude plus approfondie devrait comparer ce student à un modèle entraîné directement sans teacher, et évaluer l'impact réel sur le temps d'inférence ou sur une exécution en boucle fermée.

Enfin, les résultats obtenus avec la reformulation candidate-based doivent être lus comme complémentaires plutôt que directement comparables aux deux modèles précédents. Dans ce second cadre, la tâche, les entrées et le protocole d'évaluation changent : le modèle n'imite plus seulement une séquence de processus, mais apprend à choisir parmi des tâches candidates dans un état de scheduling plus riche. Les performances hors ligne y sont nettement plus élevées, avec 91,6% en top-1 et 99,9% en top-3, et l'évaluation en boucle fermée montre un gain de 10,5% sur le temps de réponse moyen par rapport à CFS. Ces résultats sont encourageants, car ils suggèrent qu'une formulation plus proche d'une vraie décision d'ordonnancement est potentiellement plus prometteuse qu'une simple prédiction séquentielle. Ils restent par contre exploratoires, car ils reposent sur un émulateur, un *teacher* heuristique et un cadre expérimental distinct de celui des traces réelles.

Pris dans leur ensemble, ces résultats rendent le projet intéressant et constituent une preuve de faisabilité crédible. Ils ne suffisent toutefois pas encore à confirmer scientifiquement la robustesse ou la supériorité de l'approche proposée, car le nombre d'expérimentations, la diversité des workloads et l'ampleur des validations restent encore limités. Les résultats doivent donc être lus comme très encourageants, mais encore préliminaires.

VI. LIMITES ET MENACES À LA VALIDITÉ

Les résultats obtenus restent encourageants, mais doivent être interprétés avec prudence. D'abord, une partie importante de l'entraînement repose sur des états générés sous

la politique CFS, alors que le modèle est ensuite évalué en boucle fermée sur les états qu'il induit lui-même. Ce décalage peut créer un covariate shift entre distribution d'entraînement et distribution de test. Ensuite, l'émulateur utilisé permet des comparaisons contrôlées, mais ne reproduit pas toute la complexité du scheduler Linux réel : il ne modélise pas notamment certains effets bas niveau comme les interruptions matérielles, la NUMA, les cgroups ou encore le comportement complet du noyau en production. De plus, les workloads restent synthétiques et les simulations relativement courtes, ce qui limite la portée des conclusions sur des charges réelles plus variées ou sur des horizons plus longs. Enfin, une bonne précision hors ligne ne garantit pas nécessairement une bonne qualité de décision en ligne, et les gains mesurés dépendent aussi de la qualité du teacher heuristique utilisé pour superviser l'apprentissage. Une extension naturelle serait donc de réentraîner itérativement le modèle sur les états qu'il visite lui-même en boucle fermée, dans l'esprit de DAGger [5].

VII. CONCLUSION

Ce projet montre qu'il est possible de reconstruire, à partir de traces `sched_switch`, une partie significative des décisions d'ordonnancement Linux. Le pipeline principal, centré sur un LSTM séquentiel, obtient des performances encourageantes, avec une précision top-1 supérieure à 73% et une précision top-5 supérieure à 97%. Une optimisation par sweep améliore légèrement ces résultats, et un réentraînement sur un jeu de données plus volumineux permet de dépasser 76% en top-1.

En plus de la baseline basée sur la prédiction séquentielle à partir de traces réelles, le travail introduit un second cadre méthodologique fondé sur un émulateur, dans lequel le modèle doit choisir une tâche parmi plusieurs candidates et être comparé à CFS en boucle fermée. Cette formulation rapproche davantage l'apprentissage du problème d'ordonnancement lui-même, puisqu'elle ne consiste plus seulement à reproduire une sortie observée, mais à prendre une décision à partir d'un état de scheduling plus complet.

Pris dans son ensemble, le projet est donc intéressant et les résultats obtenus sont très encourageants. En revanche, le nombre d'expérimentations, la diversité des charges et l'ampleur des validations restent encore insuffisants pour permettre une confirmation scientifiquement solide des approches proposées. À ce stade, le travail doit surtout être compris comme une preuve de faisabilité prometteuse plutôt que comme une démonstration définitive.

Une suite naturelle de ce travail serait d'améliorer la qualité du signal supervisé dans la formulation candidate-based. Le *teacher* heuristique actuel pourrait notamment être remplacé par un *teacher* appris, par exemple un modèle plus coûteux ou plus riche chargé de produire des scores de décision plus informatifs, puis distillés vers un modèle plus compact. En parallèle, il serait important de réduire le décalage entre les états vus à l'entraînement et ceux générés en boucle fermée, grâce à un réentraînement itératif sur les trajectoires induites par le modèle lui-même. Enfin, une

évaluation sur des workloads plus variés, des simulations plus longues et, à terme, un cadre noyau plus réaliste permettraient de mieux évaluer le potentiel réel de ces approches pour aller au-delà de la simple reconstruction des décisions observées.

VIII. DESCRIPTION DE L'USAGE DE L'IA GÉNÉRATIVE

Des outils d'intelligence artificielle générative, notamment ChatGPT, Gemini, Claude Code et Codex, ont été utilisés comme aides ponctuelles durant ce projet. Leur rôle a englobé un soutien méthodologique, documentaire, rédactionnel et technique.

Pour la partie logicielle, nous estimons que l'IA a généré environ 60 % du code. Claude Code et Codex ont agi comme assistants de programmation, ce qui a permis un prototypage rapide de nos idées. Grâce à ces outils, l'implémentation technique n'était plus une barrière, nous permettant de nous concentrer sur l'architecture de nos modèles et l'élaboration de nos expériences. Ils ont également aidé au débogage et à la préparation des scripts d'expérimentation.

Du côté du rapport, l'utilisation de l'IA représente environ 35 % du travail de rédaction et d'édition. ChatGPT a été utilisé pour obtenir des suggestions de structuration, unifier le style d'écriture à travers les différentes sections, améliorer la clarté de certaines formulations et optimiser la mise en page globale. De son côté, Gemini a servi à effectuer une recherche exploratoire sur les approches récentes en apprentissage profond et à identifier des références bibliographiques pertinentes, qui ont ensuite été vérifiées à partir des sources originales.

Cependant, il est important de souligner que les idées fondamentales, les choix méthodologiques, les expériences, l'analyse des résultats et les conclusions sont exclusivement issus de notre propre travail. L'ensemble des suggestions générées par l'IA a été relu, vérifié, testé et intégré de manière critique.

REFERENCES

- [1] Jingde Chen, Subho S. Banerjee, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. Machine learning for load balancing in the linux kernel. In *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '20, page 67–74, New York, NY, USA, 2020. Association for Computing Machinery.
- [2] Anusha G H, Minal K, B P Varsha, Geethishree Mishra, Meghana G K, and Madhusudhan K. N. ML engine for linux scheduler. *Quest Journals, Journal of Software Engineering and Simulation*, Volume 9 Issue 7 pp : 20-29, 2023.
- [3] Sampanna Yashwant Kahu. Kerneloracle : Predicting the linux scheduler's next move with deep learning, 2025.
- [4] Atul Negi and P. Kishore Kumar. Applying machine learning techniques to improve linux process scheduling. In *TENCON 2005 - 2005 IEEE Region 10 Conference*, pages 1–6, 2005.
- [5] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, 2011.
- [6] Vasuki Shankar. Machine learning for linux kernel optimization : Current trends and future directions. *International Journal of Computer Sciences and Engineering*, 13 :56–64, 03 2025.

ANNEXE

Liste exhaustive des IA génératives utilisées :

- 1) ChatGPT (5.3, 5.3 thinking et 5.5)
- 2) Gemini (3 rapide et 3 pro)
- 3) Claude code (Sonnet 4.6, Opus 4.7)
- 4) Codex (GPT 5.4 high)

```
#!/bin/bash
perf record -e sched:sched_switch -a -T -o dataset.data &
PERF_PID=$!

sleep 30

hackbench 20 &
sleep 30

stress-ng --cpu $(nproc) --timeout 60s

stress-ng --iomix $(nproc) --cpu 1 --timeout 60s

sleep 30

kill -INT $PERF_PID
wait $PERF_PID
```

FIG. 2. Script de génération de charge et collecte de traces avec perf

```
sudo perf script -i dataset.data > dataset_train_complet.txt
```

FIG. 3. Conversion des traces perf en fichier texte exploitable

```
Linux vps-e71cefb4 6.12.48+deb13-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.48-1 (2025-09-20) x86_64
GNU/Linux
```

FIG. 4. Informations sur le système Linux utilisé pour obtenir les traces

Exemples de prompts utilisés

- 1) “Pour chaque chiffre cité dans le rapport, trouve d’où il vient (de quel artefact) Pense à chercher dans les différentes branches”
- 2) “regarde ce que j’ai comme code avant de donner ton avis”
- 3) “Tu peux améliorer la page de couverture, elle fait très vide”
- 4) “Propose moi une reformulation pour ce paragraphe Le découpage train/validation/test suit l’ordre temporel à l’intérieur de chaque CPU avec des ratios 70/15/15. Cette décision évite une fuite temporelle simple, mais implique que le modèle est évalué sur une continuation des séquences plutôt que sur des machines différentes. Avec une fenêtre de 32 événements, on obtient 1 374 665 exemples d’entraînement, 294 471 exemples de validation et 294 472 exemples de test.”
- 5) “cherche si il y a des erreurs ou imprécisions dans rapport.tex en comparant avec tout le code des différentes branches”
- 6) “Commençons par la phase 1 : l’ingénierie et le prétraitement des données. J’ai un fichier dataset_train_complet.txt assez volumineux, environ 2 millions de lignes pour 350 Mo, contenant des traces sched_switch. Exemple de ligne : perf 1206303 [000] 6376562.594913: sched:sched_switch: prev_comm=perf prev_pid=1206303 prev_prio=120 prev_state=R+ ==> next_comm=migration/0 next_pid=21 next_prio=0. Aide-moi à construire une première version robuste du script, en gardant le code facile à adapter et en explicitant les hypothèses. L’objectif est de parser les traces pour obtenir des données exploitables pour l’entraînement, avec notamment : extraction des champs utiles : timestamp, cpu, prev_comm, prev_pid, prev_prio, prev_state, next_comm, next_pid; séparation

des événements par CPU pour ne pas mélanger les historiques; calcul d'un `delta_t` par CPU; encodage des processus à partir de `prev_comm / next_comm`, avec un vocabulaire des processus fréquents et un identifiant `<UNK>` pour les processus rares; approche performante, idéalement avec `polars` ou `pandas`.”

- 7) “Le dernier merge a causé des erreurs. Peux-tu analyser les logs pour identifier la cause racine, proposer un fix, puis simuler un re-run pour vérifier la cohérence du code”